

System Design & Domain Note: LLD Practice Platform MVP

Author: Candidate Engineering Submission **Focus:** Domain Design, Object Responsibilities, Evaluation Architecture & Trade-offs ---

1. System Overview & Learner Journey

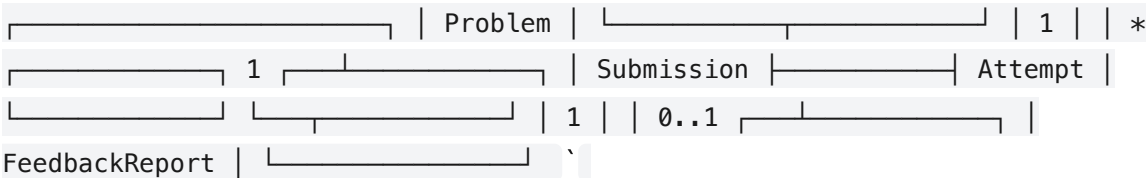
The LLD Practice Platform provides an end-to-end environment for practicing object-oriented design. `` [Problem Catalog] —> [Design Workspace] —> [Async Submission] | [Attempt Progression] <— [Feedback Dashboard] <— [Evaluation Pipeline] `

The Practice Loop:

1. **Choose Problem:** Select from curated LLD problems (e.g., Parking Lot, Elevator Controller, Vending Machine, Rate Limiter). 2. **Design & Implement:** Use the multi-tab studio workspace to write class contracts, define responsibility mappings, detail concurrency assumptions, and sketch visual diagrams. 3. **Submit Solution:** Non-blocking async API endpoint stores submission and triggers evaluation pipeline. 4. **Hybrid Evaluation:** Deterministic static analysis runs alongside LLM architectural reasoning against a 6-axis rubric. 5. **Review Feedback:** Review overall score gauge, evidence citations, strengths vs. anti-patterns, and extensibility stress-test hints. 6. **Iterate (Try Again):** Submit Attempt #2 and view side-by-side progression diffs. ---

2. Object Model & Class Responsibilities

The domain is modeled cleanly adhering to Object-Oriented Design principles: `



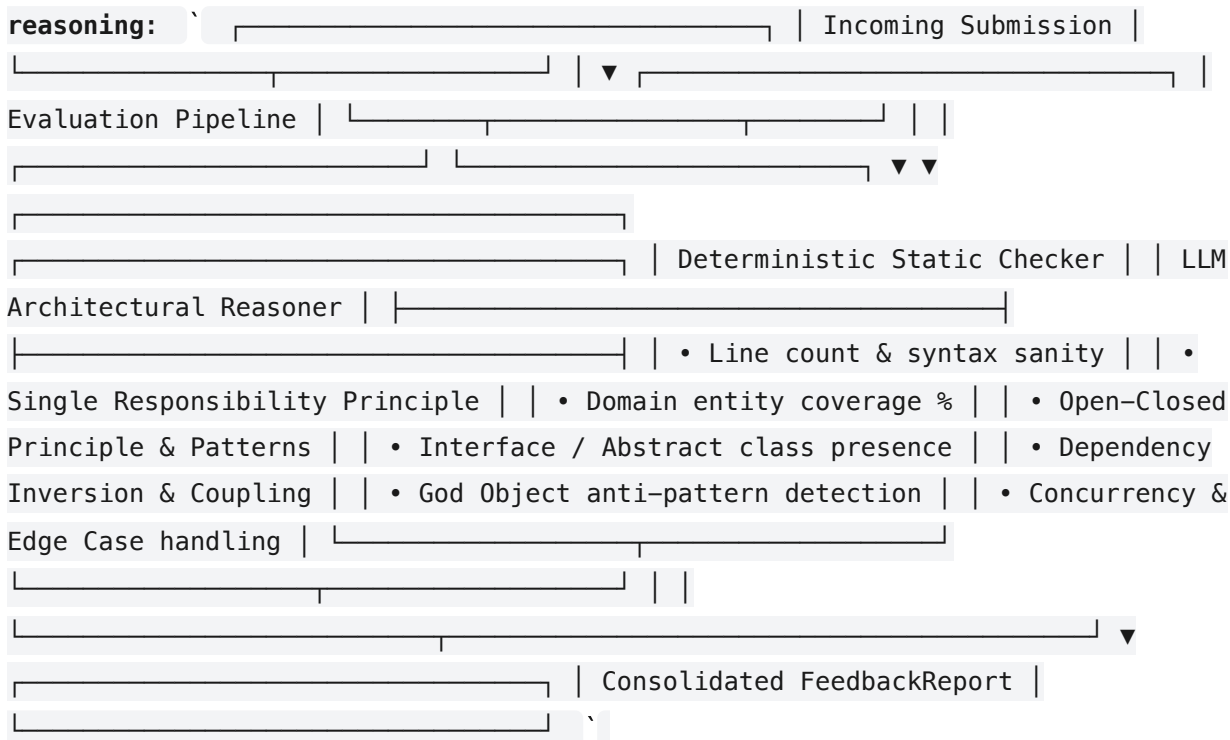
Core Domain Entities:

Entity	Type	Primary Responsibility
--------	------	------------------------

Problem	Entity	Owns problem requirements, constraints, expected domain concepts, starter templates, and evaluation rubrics.
Submission	Value Object	Encapsulates candidate solution (language, code, responsibility mappings, trade-off notes, diagram syntax).
Attempt	Aggregate Root	Manages state machine lifecycle (DRAFT $\rightarrow$ SUBMITTED $\rightarrow$ EVALUATING $\rightarrow$ COMPLETED / FAILED), attempt sequence numbering, and timestamp audit logs.
Rubric	Value Object	Defines 6 evaluation criteria, weight distribution, and scoring guides.
FeedbackReport	Value Object	Represents immutable evaluation results, overall score, rubric breakdowns, evidence citations, strengths, anti-patterns, and extensibility hints.
EvaluationPipeline	Composite Domain Service	Coordinates registered <code>IEvaluator</code> strategies to compute composite scores and feedback.
DeterministicEvaluator	Strategy Implementation	Performs static analysis (structural volume, entity coverage, interface presence, god-object detection).
LLMEvaluator	Strategy Implementation	Executes structured architectural reasoning (SOLID compliance, coupling, concurrency, trade-offs) with automatic fallback.

3. Evaluation Approach: Deterministic vs. LLM

Evaluation separates **objective structural validation** from **subjective architectural reasoning**:



Why Fixed Rubrics & Structured Schema?

To prevent unconstrained LLM hallucinations (e.g. giving 100/100 to an empty class), the `LLMEvaluator` enforces a strict JSON schema where every score **must** cite specific evidence from the user's code or explanation. ---

4. Architectural Resilience: Change Tests

Change Test A: Supporting Diagram / Visual Submissions

> Today the learner submits text/code. Later the platform supports Class Diagrams (Mermaid/PlantUML). **Domain Proof:** Our `Submission` value object includes an optional `diagramSyntax?: string` property.

- The `Attempt` aggregate root and `EvaluationPipeline` contract remain **completely untouched**.

- A new `DiagramEvaluator` can be plugged into `EvaluationPipeline` without changing any practice loop code.

Change Test B: Adding New Evaluators (e.g., Human Review / Static Analyzers)

> Today feedback comes from static + LLM evaluators. Later you add SonarQube or Human Peer Review. **Domain Proof:** We implemented the **Strategy Pattern** via the `IEvaluator` interface:

```
`typescript export interface IEvaluator { name: string; evaluate(problem: Problem, submission: Submission): Promise; }`
```

 The `EvaluationPipeline` provides a `registerEvaluator(evaluator: IEvaluator)` method. Adding a `HumanPeerEvaluator` or `StaticSecurityEvaluator` requires zero changes to the core submission flow. ---

5. Practical Scaling & Failure Trade-offs

1. Non-Blocking Async Processing: Submissions are persisted immediately with status `SUBMITTED`. Evaluation is dispatched asynchronously via background event loops (`setImmediate`). The client receives a `202 Accepted` response and polls status, preventing HTTP timeouts if LLM APIs take 3–5 seconds. **2. Resilient LLM Fallback:** If the external Gemini/OpenAI API is unreachable or rate-limited, `LLMEvaluator` seamlessly falls back to an internal heuristic reasoning engine. Evaluation **never fails** due to external API outage. **3. Storage & Monolith Simplicity:** The prototype uses JSON disk persistence (`attempts_store.json``), eliminating database setup dependencies while providing full persistence across process restarts.